# Answer key | MCQs and short FRQs

## Quick key

```
01 C   02 A   03 A   04 D   05 B   06 D   07 D   08 A

09 C   10 B   11 B   12 D   13 A   14 A   15 D   16 D

17 C   18 A   19 A   20 D   21 A   22 A   23 C   24 B
```

## How to review

For each missed trace, identify the first mistaken step: argument binding, scope, mutation, return, or caller assignment. For a guessed answer, explain why the other options are wrong before marking it reviewed.

## Short-answer scoring

Each FRQ has a suggested 6-point rubric. Equivalent correct solutions receive credit. This is a practice rubric, not an official course grading scheme.

## Coding checklist

Each challenge: correct return structure, all stated edge cases, permitted operations, preserved or intentionally mutated inputs, and a meaningful docstring. Use the two hints in order before reading the reference solution.

# MCQ explanations | Q1-Q4

### Q1 | C. Read sessions; answer queries; report results. Then refine each task.

Start with the overall goal, identify coherent major responsibilities, and refine them until implementation is clear. Equal size or shared syntax does not make a coherent task.

Source: L8 pp11-18

---

### Q2 | A. B, C, and D are leaves; A is the parent of C and D.

A leaf has no children, even when it is closer to the root than other leaves. The root has no parent, not no children.

Source: L8 p13

---

### Q3 | A. Several decompositions can be reasonable; hierarchies may miss cross-connections.

The lecture allows multiple designs and notes artificial boundaries and uncaptured relationships. Stop when implementation is obvious, not at a rigid line count.

Source: L8 pp14-18

---

### Q4 | D. 3 4 9

Updating an existing key does not add a pair. C stores 9 when assigned; changing B afterward does not recalculate C.

Source: L8 pp28-32

---

# MCQ explanations | Q5-Q8

### Q5 | B. "ab" in d and 3 not in d

Dictionary membership checks exact keys. It neither searches substrings inside keys nor searches values. Repeated values are allowed.

Source: L8 pp28,31-32

---

### Q6 | D. Prints 7 9, then raises KeyError

Integer 0 and string "0" are different keys. d[0] is a lookup by key, not a request for the first entry.

Source: L8 pp29-31

---

### Q7 | D. 6 2 2

Initialize each key only on its first encounter. Every occurrence adds 2. len counts distinct keys, not iterations.

Source: L8 pp29-32 plus L6 loops

---

### Q8 | A. 8 3 5

The loop variable receives keys. Updating values for existing keys is allowed; the updated values 3 and 5 contribute to total.

Source: L8 pp30-32

---

# MCQ explanations | Q9-Q12

### Q9 | C. 5

A definition does not execute its body. The undefined function is inside an uncalled body, so it is never looked up.

Source: L9 pp39-51

### Q10 | B. 6 / None

The inner print displays 6. Because double reaches its end without return, its returned value is None, which is assigned to y and then printed.

Source: L9 pp10-13,32-36,81

### Q11 | B. 8 -1

return exits the entire function at the first match. The final return handles the no-match case only after the loop finishes.

Source: L9 pp81-84 plus L6

### Q12 | D. 3 2 5

The function returns one tuple (3,2). Unpacking follows position, regardless of whether the caller uses conventional quotient/remainder names.

Source: L9 pp14-21

# MCQ explanations | Q13-Q16

### Q13 | A. 7 13 12

Provided positional arguments fill from left to right; omitted trailing arguments use their defaults. Zero is supplied explicitly, so it does not trigger a default.

Source: L9 pp85-88

---

### Q14 | A. def f(a=1, b):

A required positional parameter cannot follow one with a default. This is a syntax error in the definition, not an error postponed until calling.

Source: L9 pp85-88

---

### Q15 | D. 5 8 11

Each call has its own parameter binding. Reassigning a numeric parameter does not change the caller variable. The returned value matters only where it is used.

Source: L9 pp65-78,81

---

### Q16 | D. 12

With no local rate assignment, the function reads the global rate when called. It does not freeze rate at definition time. Passing rate as an argument would make the dependency clearer.

Source: L9 pp63-64

---

# MCQ explanations | Q17-Q20

### Q17 | C. UnboundLocalError before any output

+= is assignment. Without a parameter, local initialization, or global declaration, score is treated as local but read before receiving a value.

Source: L9 p77

---

### Q18 | A. 3 5 5

global allows assignment to the main-program count. The first return stores 3 in a; the later change to count does not alter a. Prefer explicit parameters and returns in new designs.

Source: L9 pp79-80

---

### Q19 | A. [9, 2] [4, 8]

The first element assignment mutates the shared original list. Rebinding local a to a new list separates later changes; returning that new list gives it to y.

Source: L9 pp97-106

---

### Q20 | D. [1, 7] [2]

For lists, += extends the shared object. b + [8] builds a new list and b = rebinds only the local parameter.

Source: L9 pp90-98

---

# MCQ explanations | Q21-Q24

### Q21 | A. [6, 2] [6, 2] [1, 9]

a and b refer to one list. The slice makes a separate outer list; its numeric entries can be replaced independently.

Source: L9 p107

### Q22 | A. Prints [9, 2], then raises TypeError

Tuple slots cannot be reassigned, but a list referenced by a slot remains mutable. The tuple still refers to the same list after its contents change.

Source: L9 pp18,99

### Q23 | C. 17

g(3) calls f(3), gets 5, and returns 5*3=15. The outer f then receives 15 and returns 17. Keep a separate x for each invocation.

Source: L9 pp38-50,65-78

### Q24 | B. 3 1 3 2

The passed dictionary is mutable. Both updates affect counts, including the new key. Each returned integer is stored separately.

Source: L8 pp29-32; L9 p96

# Short FRQ answers | F1-F2

## F1 | Design before coding

One valid tree: Study analyzer -> Read sessions (read name; validate/read minutes); Answer queries (find matching sessions; calculate count/total); Present final summary (calculate all-session totals; format report). Read sessions is a parent; read name is a leaf. Refine until implementation is clear in a few lines. Other coherent, distinct decompositions are valid.

Rubric: 2 points: coherent major tasks. 2: meaningful refinement. 1: correct parent/leaf identification. 1: stopping criterion.

Source: L8 pp11-22

---

## F2 | Repair dictionary counting

The count resets on every occurrence, and return exits during the first iteration. Initialize only missing keys and return after the loop.

```
def count_words(words):
    counts = {}
    for word in words:
        if word not in counts:
            counts[word] = 0
        counts[word] += 1
    return counts
```

Rubric: 2 points: identify both defects. 2: correct initialization/update. 1: return after loop. 1: empty list returns {}.

Source: L8 pp29-32

---

# Short FRQ answers | F3-F4

## F3 | Separate print from return

It prints 12, then raises TypeError because area returns None and None + 1 is invalid. Return the product instead.

```
def area(w, h):
    """Return rectangle area for width w and height h."""
    return w * h
result = area(3, 4) + 1
```

Rubric: 2 points: 12 then TypeError. 2: return product. 1: meaningful docstring. 1: caller obtains 13 without function output.

Source: L9 pp10-13,81-84

---

## F4 | Parameter order and tuple results

The default call may use any valid total. split_total(17, 5) returns (3, 2). Unpacking uses the returned order.

```
def split_total(total, people=2):
    return total // people, total % people
share, leftover = split_total(9)
```

Rubric: 1 point: correct default header. 2: // and %. 1: tuple return. 1: default call/unpacking. 1: (3, 2).

Source: L9 pp14-21,85-88

---

# Short FRQ answers | F5-F6

## F5 | Repair scope without global

Assignment makes score local to increase, but += reads it before any local value exists: UnboundLocalError. Pass the value, return the new value, and assign that return in the caller.

```
def increase(score):
    return score + 2
score = 10
score = increase(score)
```

Rubric: 2 points: scope explanation. 1: parameter. 1: return. 1: caller assignment. 1: no global.

Source: L9 pp63-80

---

## F6 | Trace two references

Initially x ends as [5,4] and y as [5,4,9]. a[0] += 3 mutates the shared list; a = a + [9] rebinds the local parameter to a new list. Change the call to y = edit(x[:]) to protect the original list of numbers.

Rubric: 2 points: final values. 2: mutation versus rebinding. 2: slice at call and correct unchanged x.

Source: L9 pp93-107

---

# Short FRQ answers | F7-F8

## F7 | Early return or full scan?

return inside the loop stops after the first item, and an empty list reaches the end without a return. Move return count to the same indentation level as for. The corrected results are 2 and 0.

```
def tally(values, threshold):
    count = 0
    for value in values:
        if value >= threshold:
            count += 1
    return count
```

Rubric: 2 points: both failure modes. 2: indentation repair. 2: correct results.

Source: L9 pp81-84; L6 loops

---

## F8 | Explain tuple immutability precisely

Output: A [2, 7]. The tuple keeps its reference to the original string "A"; rebinding name does not replace the tuple slot. It also keeps a reference to scores, whose contents can change. t[1] = [0] would raise TypeError because it replaces a tuple slot.

Rubric: 2 points: output. 2: distinguish string rebinding and list mutation. 2: reject tuple item assignment with explanation.

Source: L9 pp18,99

---

# Coding C1 | Hints and solution

## Study-session query engine

### Hint 1

Use each test name as a key and store [count, total] as its value. Initialize a missing name before updating it.

### Hint 2

Build a separate output list by looping through queries. For missing names append a zero-result tuple; do not add missing queries to the session data.

```
def analyze_sessions(names, minutes, queries):
    """Return count and total minutes for each query."""
    totals = {}
    for i in range(len(names)):
        name = names[i]
        if name not in totals:
            totals[name] = [0, 0]
        totals[name][0] += 1
        totals[name][1] += minutes[i]
    results = []
    for name in queries:
        if name in totals:
            results.append((name, totals[name][0], totals[name][1]))
        else:
            results.append((name, 0, 0))
    return results
```

### Why it works

Each input session contributes exactly once. A zero-minute session still counts as one session. Iterating over queries preserves order and duplicates. All modified lists belong to the newly created totals/results structures, so the inputs remain unchanged.

Source: L8 pp19-25,28-32; L9 pp14-21,65-81

# Coding C2 | Hints and solution

## First unique reading

### Hint 1

In your first pass, count every value with a dictionary. A value seen once so far may appear again later.

### Hint 2

In a second pass, visit original indices in increasing order. Return at the first count of one; put the no-match return after the loop.

```
def first_unique(values):
    """Return the first globally unique reading and its index."""
    counts = {}
    for value in values:
        if value not in counts:
            counts[value] = 0
        counts[value] += 1
    for i in range(len(values)):
        if counts[values[i]] == 1:
            return i, values[i]
    return -1, 0
```

### Why it works

The first pass establishes global frequencies before any uniqueness decision. The second pass enforces earliest original index. The returned index distinguishes a real unique zero (0,0) from no result (-1,0).

Source: L8 pp28-32; L9 pp19-21,65-81

# Coding C3 | Hints and solution

## Sequential stock requests

### Hint 1

Choose the working dictionary first: use stock itself for apply=True, or build a new dictionary with for key in stock for simulation.

### Hint 2

Check membership before accessing an item. Append one Boolean per request. Only accepted requests subtract stock, and every later request uses the updated working balance.

```
def process_requests(stock, names, amounts, apply=False):
    """Return acceptance flags and real or simulated ending stock."""
    if apply:
        working = stock
    else:
        working = {}
        for key in stock:
            working[key] = stock[key]
    accepted = []
    for i in range(len(names)):
        name = names[i]
        amount = amounts[i]
        ok = False
        if name in working:
            if working[name] >= amount:
                working[name] -= amount
                ok = True
        accepted.append(ok)
    return accepted, working
```

### Why it works

Rebinding working = stock intentionally shares the dictionary. The simulation loop copies integer quantities into a distinct dictionary. Repeated requests see earlier accepted withdrawals. A rejected request does not reserve stock. Exact equality is sufficient for acceptance.

Source: L8 dictionary operations; L9 defaults, tuple returns, mutation/rebinding